

Deep Learning for Face Editing and Face Generation

Deep Learning Project Presentation

Azammat CHARAF ZADAH, Youcef BOUZID, Martin CUINGNET

github.com/mcngnt/face_gen_dl_project

2025-05-19

Project's Ambition

Goal : Manipulate face information :

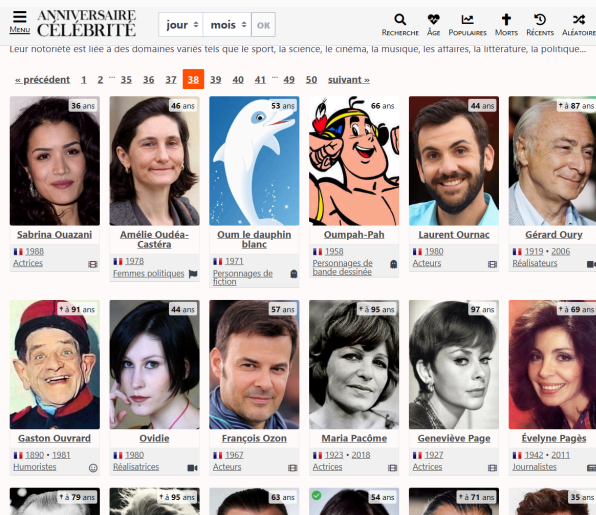
- Face interpolation
- Recreate faces
- Face editing
- Finding your lookalike

How ? : By representing faces in the latent space of a model

Datasets

Two Datasets

- **Anniversaire-Célébrités** : 6 507 faces scrapped from <https://anniversaire-celebrite.com/>
Useful to find unanonymized lookalike

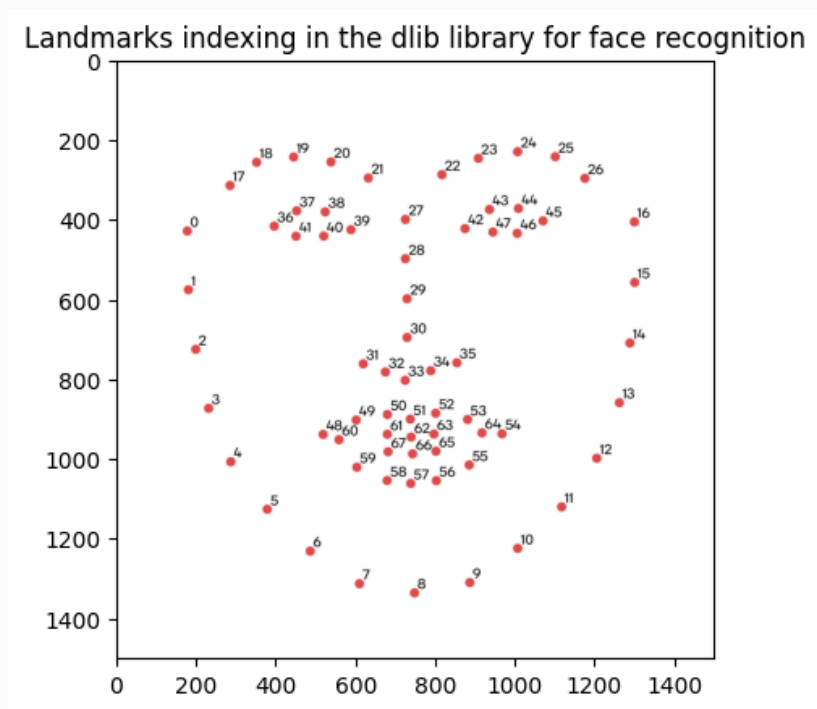


Screenshot of the website

- **CelebA** : *More common dataset, with features pre-computed*

Processing

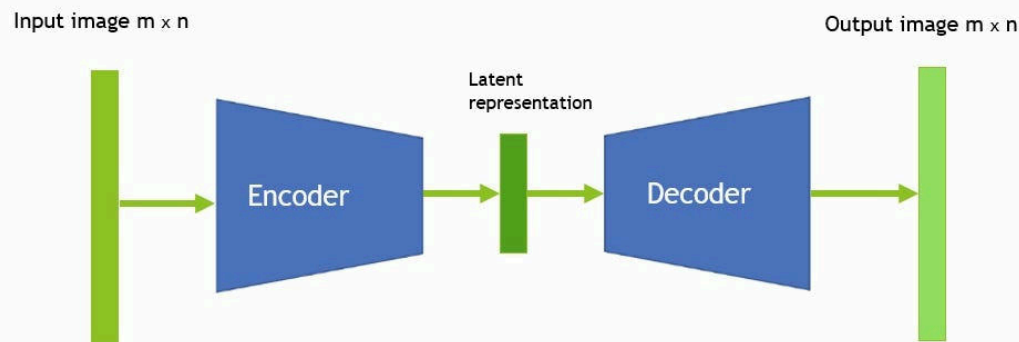
- **Face alignment** (also used to remove non-human faces)



- **Cropping** to square format
- **Downsampling** (converting to 128x128 for instance)

AutoEncoder

Architecture



General Architecture

```
self.encoder = nn.Sequential(  
    nn.Conv2d(3, 64, kernel_size=4, stride=2, padding=1),  
    nn.ReLU(),  
    nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1),  
    nn.ReLU(),  
    nn.Conv2d(128, 256, kernel_size=4, stride=2,  
padding=1),  
    nn.ReLU(),  
    nn.Flatten(),  
    nn.Linear(256 * 8 * 8, feature_dim))
```

```
self.fc = nn.Linear(feature_dim, 256 * 8 * 8)  
self.decoder = nn.Sequential(  
    nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2,  
padding=1),  
    nn.ReLU(),  
    nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2,  
padding=1),  
    nn.ReLU(),  
    nn.ConvTranspose2d(64, 3, kernel_size=4, stride=2,  
padding=1),  
    nn.Sigmoid())
```

- On 64x64 and 128x128 images
- Using epochs between 10 and 20
- 100 or 300 dimensional latent space
- $\gamma = 0.001$, `batch_size` = 16
- Adam Optimizer with MSE Loss

First Results and Lerping



Random faces sampled from PCA



Reconstruction of Unseen Image

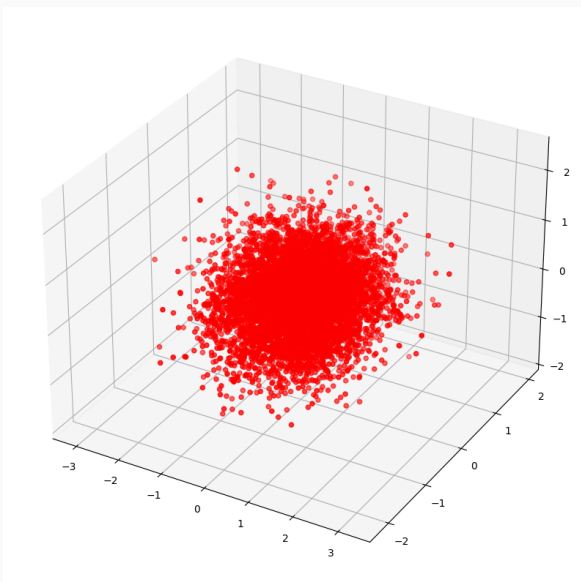


Mixing Trump and Obama

Face Editing

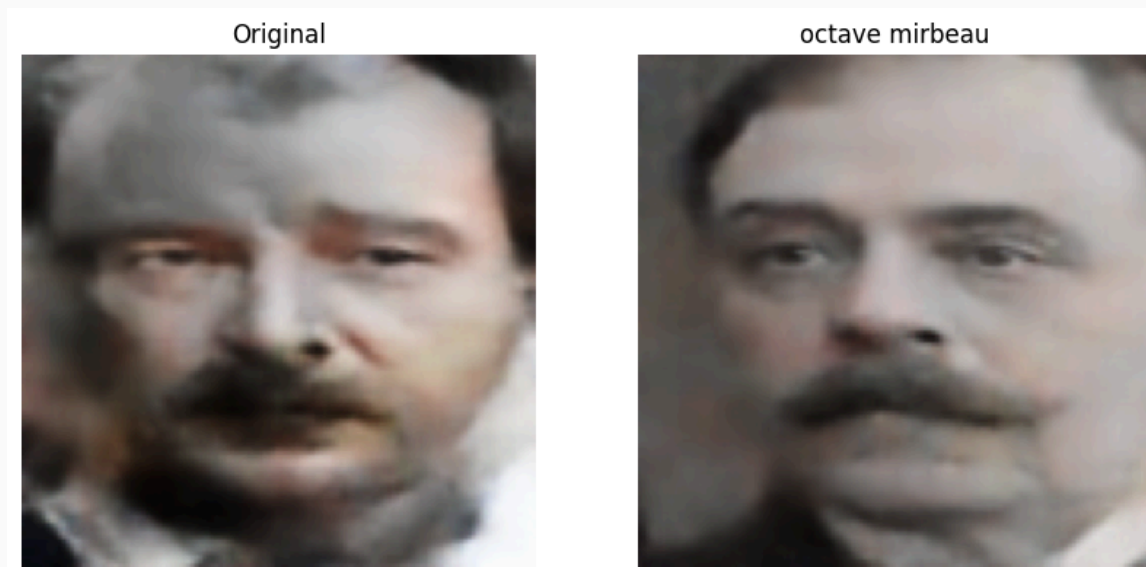
PCA and Random Face

PCA on the embedding of every face in the dataset



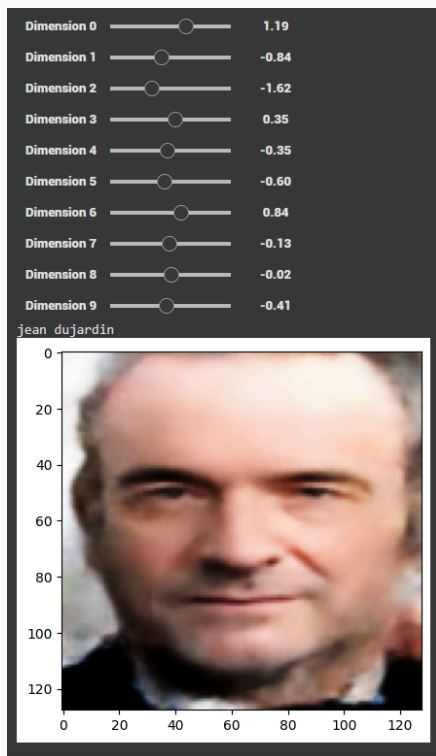
PCA projection in 3 dimensions

Random face as well as its closest celebrity



Random Face and its closest celebrity look-alike

Face Editing



Face editing with PCA parameters

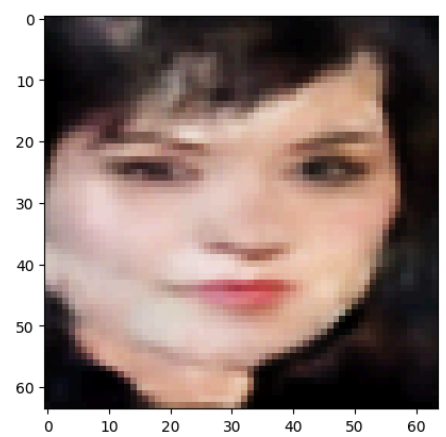
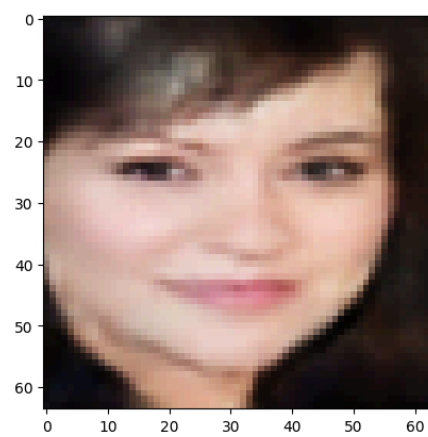
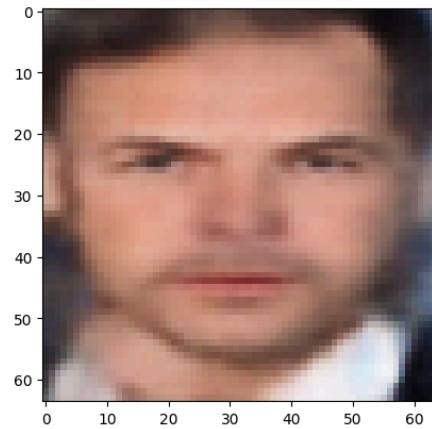
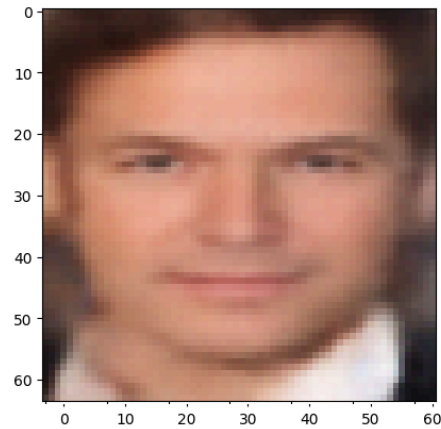
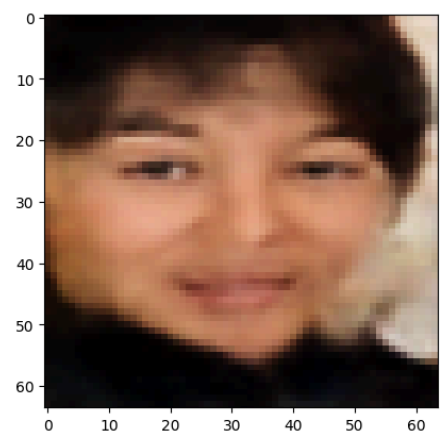
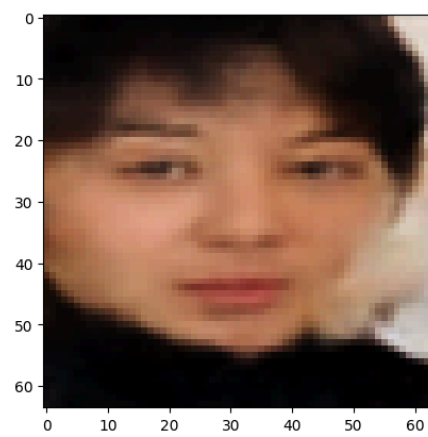
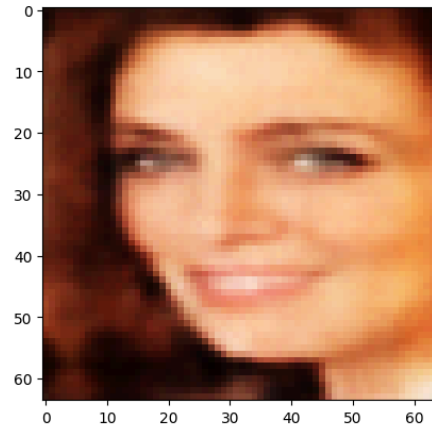
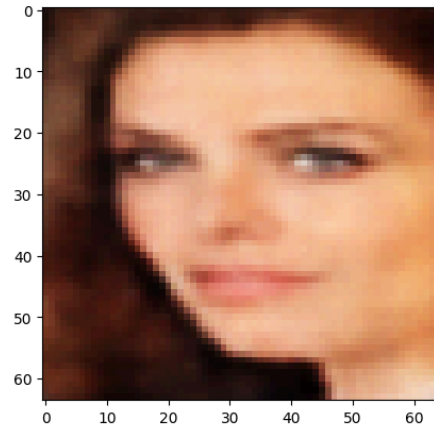
0. Hair darkness
1. Side lighting / Face orientation
2. Age
3. Skin Color
4. Saturation (from yellow to black and white)
5. Age
6. Facial Pilosity (Hair and beard)
7. Saturation + Age
8. Face Orientation
9. Gender

Using CelebA 1/2

Use the **pre-computed features** of CelebA to find the direction of the feature in latent-space with a linear classifier.

Index	Definition	Index	Definition	Index	Definition	Index	Definition
1	5o'ClockShadow	11	Blurry	21	Male	31	Sideburns
2	ArchedEyebrows	12	BrownHair	22	MouthSlightlyOpen	32	Smiling
3	Attractive	13	BushyEyebrows	23	Mustache	33	StraightHair
4	BagsUnderEyes	14	Chubby	24	NarrowEyes	34	WavyHair
5	Bald	15	DoubleChin	25	NoBeard	35	WearingEarrings
6	Bangs	16	Eyeglasses	26	OvalFace	36	WearingHat
7	BigLips	17	Goatee	27	PaleSkin	37	WearingLipstick
8	BigNose	18	GrayHair	28	PointyNose	38	WearingNecklace
9	BlackHair	19	HeavyMakeup	29	RecedingHairline	39	WearingNecktie
10	BlondHair	20	HighCheekbones	30	RosyCheeks	40	Young

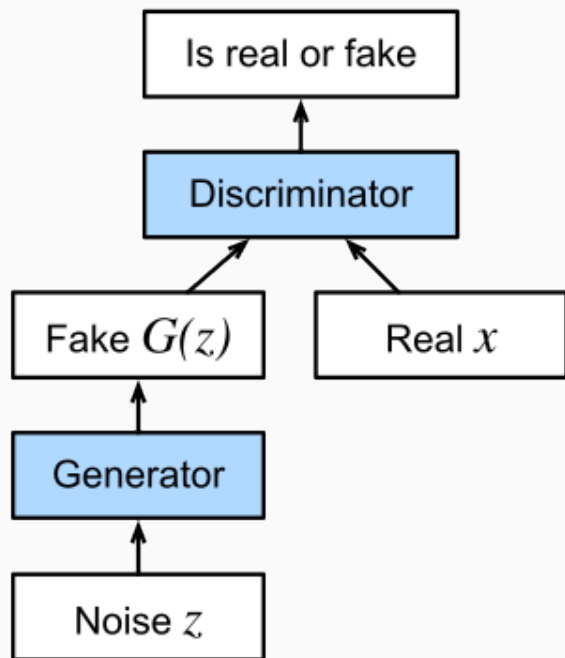
Using CelebA 2/2



GAN



GAN definition



- Made by Goodfellow et al.
- Use of a generative and discriminative model
- Improve both models competitively

Minimax problem between G and D: $\min_G \max_D (V(G, D))$ where:

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log(D(x))] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

GAN implementation 1/2

```
class Generator(nn.Module):
    def __init__(self, z_dim, channel_img, feature_g):
        super().__init__()
        self.gen = nn.Sequential(
            self._block(z_dim, feature_g*16, 4, 2, 0),
            self._block(feature_g*16, feature_g*8, 4, 2, 1),
            self._block(feature_g*8, feature_g*4, 4, 2, 1),
            self._block(feature_g*4, feature_g*2, 4, 2, 1),
            nn.ConvTranspose2d(feature_g*2, channel_img, 4, 2, 1),
            nn.Tanh(),
        )

    def _block(self, in_channels, out_channels, kernel_size,
stride, padding):
        return nn.Sequential(
            nn.ConvTranspose2d(in_channels, out_channels,
kernel_size, stride, padding),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(),
        )

    def forward(self, x):
        return self.gen(x)
```

```
class Critic(nn.Module):
    def __init__(self, channel_img, feature_d):
        super().__init__()
        self.disc = nn.Sequential(
            nn.Conv2d(channel_img, feature_d, kernel_size=4,
stride=2, padding=1),
            nn.LeakyReLU(0.2),
            self._block(feature_d, feature_d*2, 4, 2, 1),
            self._block(feature_d*2, feature_d*4, 4, 2, 1),
            self._block(feature_d*4, feature_d*8, 4, 2, 1),
            nn.Conv2d(feature_d*8, 1, 4, 2, 0),
        )

    def _block(self, in_channels, out_channels, kernel_size,
stride, padding):
        return nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size,
stride, padding, bias=False),
            nn.InstanceNorm2d(out_channels, affine = True),
            nn.LeakyReLU(0.2),
        )

    def forward(self, x):
        return self.disc(x)
```

GAN implementation 2/2

```
criterion = nn.BCELoss()
optimizer_G = torch.optim.Adam(generator.parameters(), lr=lr, betas=(0.5, 0.999))
optimizer_D = torch.optim.Adam(discriminator.parameters(), lr=lr, betas=(0.5, 0.999))

for epoch in range(epochs):
    for i, real_images in enumerate(dataloader):
        real_images = real_images.to(device)
        batch_size = real_images.size(0)

        real_labels = torch.ones(batch_size, 1).to(device)
        fake_labels = torch.zeros(batch_size, 1).to(device)

        optimizer_D.zero_grad()
        outputs = discriminator(real_images)
        outputs = outputs.view(-1, 1)
        d_loss_real = criterion(outputs, real_labels)
        d_loss_real.backward()

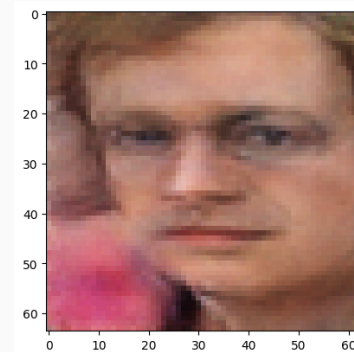
        z = torch.randn(batch_size, latent_dim, 1, 1).to(device)
        fake_images = generator(z)

        outputs = discriminator(fake_images.detach())
        outputs = outputs.view(-1, 1)
        d_loss_fake = criterion(outputs, fake_labels)
        d_loss_fake.backward()

        optimizer_D.step()
        d_loss = d_loss_real + d_loss_fake

        optimizer_G.zero_grad()
        outputs = discriminator(fake_images)
        outputs = outputs.view(-1, 1)
        g_loss = criterion(outputs, real_labels)
        g_loss.backward()
        optimizer_G.step()
```

Random Face Generation - GAN



WGAN(GP)

```
for epoch in range(num_epochs):
    for batch_idx, (real, _) in enumerate(loader):
        real = real.to(device)

        for _ in range(critic_iterations):
            noise = torch.randn(batch_size, z_dim, 1,
1).to(device)
            fake = gen(noise)

            # G(z)

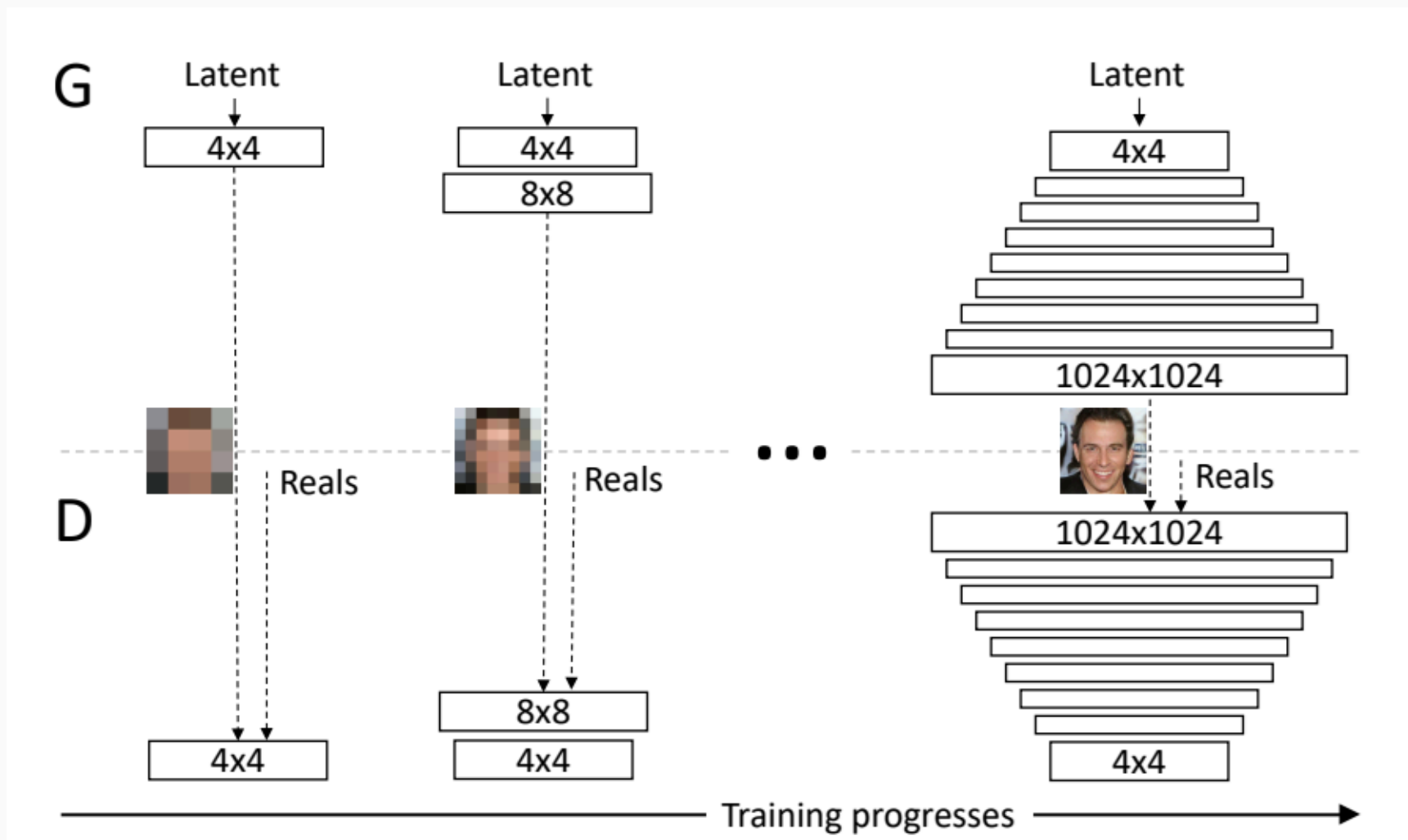
            critic_real = critic(real).reshape(-1)
            critic_fake = critic(fake).reshape(-1)
            gp = gradient_penalty(critic, real, fake)
            loss_critic = -(torch.mean(critic_real) -
torch.mean(critic_fake)) + lambda_GP*gp
            #loss_crit.append(loss_critic)
            critic.zero_grad()
            loss_critic.backward(retain_graph = True)
            opt_critic.step()

            # Training Generator : -min E(critic(fgen_fake))
            output = critic(fake).reshape(-1)
            loss_gen = -torch.mean(output)
            #loss_generator.append(loss_gen)
            gen.zero_grad()
            loss_gen.backward()
            opt_gen.step()
```

Random Face Generation - WGAN with Gradient Penalty



Progressive Growing of GAN



Growing GAN implementation (fail)

```
def forward(self, input, step=0, alpha=-1):
    if step > self.max_step:
        step = self.max_step

    out_4 = self.input_layer(input.view(-1, self.input_dim, 1, 1))
    out_4 = self.progression_4(out_4)
    out_8 = self.progress(out_4, self.progression_8)
    if step==1:
        if self.tanh:
            return torch.tanh(self.to_rgb_8(out_8))
        return self.to_rgb_8(out_8)

    out_16 = self.progress(out_8, self.progression_16)
    if step==2:
        return self.output( out_8, out_16, self.to_rgb_8, self.to_rgb_16, alpha )

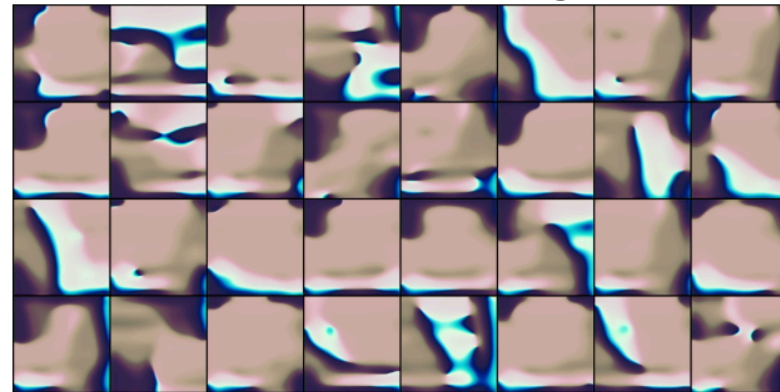
    out_32 = self.progress(out_16, self.progression_32)
    if step==3:
        return self.output( out_16, out_32, self.to_rgb_16, self.to_rgb_32, alpha )

    out_64 = self.progress(out_32, self.progression_64)
    if step==4:
        return self.output( out_32, out_64, self.to_rgb_32, self.to_rgb_64, alpha )

    out_128 = self.progress(out_64, self.progression_128)
    if step==5:
        return self.output( out_64, out_128, self.to_rgb_64, self.to_rgb_128, alpha )

    out_256 = self.progress(out_128, self.progression_256)
    if step==6:
        return self.output( out_128, out_256, self.to_rgb_128, self.to_rgb_256, alpha )
```

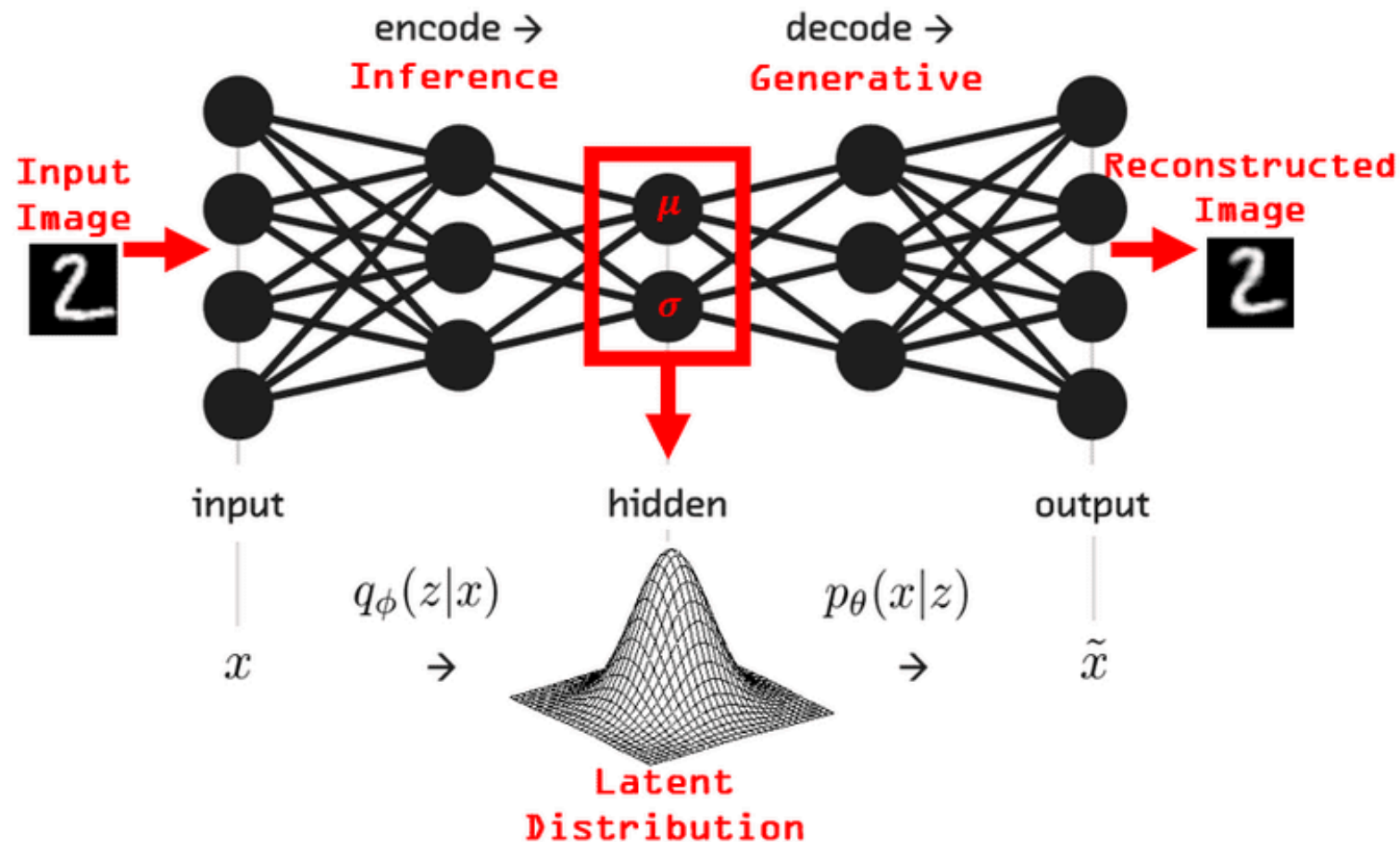
Random Face Generation Failed - Progressive GAN



VAE



Architecture



VAE architecture - Produce Distribution in Latent Space

```
class VAE128(nn.Module):
    def __init__(self, latent_dim):
        super(VAE128, self).__init__()
        self.encoder = VEncoder128(latent_dim)
        self.decoder = VDecoder128(latent_dim)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5*logvar)
        eps = torch.randn_like(std)
        return mu + eps*std

    def encode(self, x):
        mu, logvar = self.encoder(x)
        return mu, logvar

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon_x = self.decode(z)
        return recon_x, mu, logvar
```

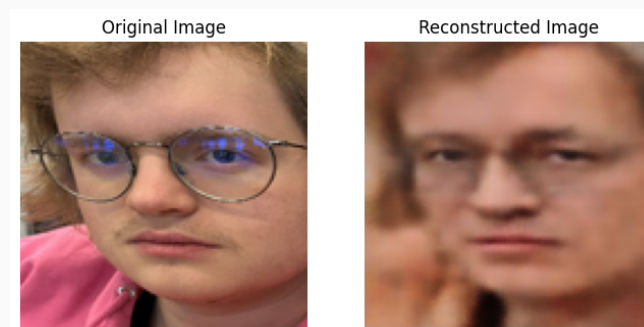
```
def vae_loss(x, x_hat, mean, log_var):
    reproduction_loss =
nn.functional.binary_cross_entropy(x_hat, x,
reduction='sum')
    KLD = - 0.5 * torch.sum(1+ log_var - mean.pow(2)
- log_var.exp())

    return reproduction_loss + KLD
```

Results



Random faces sampled from PCA

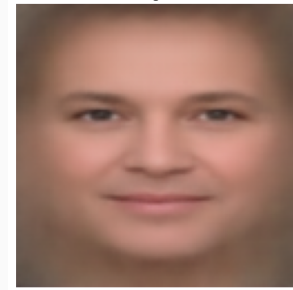


Reconstruction of Unseen Image

Conclusion

Our approach is
working but still a few
limitations :

- Small Resolution /
Lack of details



Average
feature
face

- Lack of control and interpretability when editing faces
- Small and bisased dataset

(anniversaire-
célébrités)

What could be **improved** :

- Working progressive scaling model

- Try β -VAE to achieve a disentangled representation (and thus better interpretability)
- Supervised learning with predetermined features

Thank you for your attention !